

Adaptive Conservatism Selection in Offline-to-Online Reinforcement Learning for Portfolio Allocation

Thomas Lee UC Berkeley tholee@berkeley.edu

May 10, 2026

Extended Abstract

We study reinforcement learning for portfolio allocation across eight broad-market ETFs at daily frequency: the agent emits weights on the simplex and the env rewards the post-cost log return $r_t = \log(1 + w_t^\top R_{t+1} - \lambda \|w_t - w_{t-1}\|_1)$. Our milestone established that the standard offline-RL stack collapses smoothly under realistic transaction-cost friction on this universe: offline-trained policies fail to reach the equal-weight baseline at $\lambda=0.001$ and actively destroy capital at $\lambda=0.005$. This paper asks whether offline-to-online (O2O) fine-tuning with adaptive conservatism scheduling repairs the collapse. The headline empirical result, on a 1,061-day causal test window (2022-Q1 to 2026-Q1, the post-2022 stock-bond-correlation regime), is yes: an adaptive-O2O agent (Geodesic-CQL pretrain on 2008–2020 transitions, SAC-Dirichlet fine-tune with $\beta_t = \alpha_{\text{CQL}} \cdot \sigma(\eta \cdot \text{KL}(h_{\text{off}} \| h_{\text{on}}))$) achieves test Sharpe 1.762 ± 0.154 across three seeds — the only RL method in our study to beat both equal-weight (0.953) and risk-parity (1.161) on this window.

The mechanism behind that headline is not what we pre-registered. Our pre-registered hypothesis was that the modulated β_t would let the SAC fine-tune trade actively in response to distribution shift in the test window. The Phase 2C results disconfirm that mechanism. Both naive ($\beta_t \equiv \alpha_{\text{CQL}}$) and adaptive O2O converge to functionally static allocations (per-seed turnover $\leq 2.2 \times 10^{-4}$ across all six runs, three orders of magnitude below GRPO online’s 0.24); the differential between them is not how they trade but which static allocation the SAC fine-tune lands on. Naive O2O’s pinned conservatism leaves the basin random across seeds (Sharpe range $[-0.236, +2.179]$, std 1.21); adaptive O2O’s modulated schedule consistently steers the same SAC dynamics from the same Geodesic-CQL pretrain to a uniformly strong basin (Sharpe range $[+1.629, +1.931]$, std 0.15, an eight-fold variance reduction). The conservatism schedule’s load-bearing function in our results is convergence-basin steering during fine-tune, not online responsiveness during deployment.

The four-way O2O comparison sits inside a broader failure-mode landscape, with each mode mechanistically traceable. (i) On a leaky offline pipeline (in-observation reward feature, since fixed; Appendix B), Q -maximizing offline RL exploits the leak: TD3+BC test Sharpe 6.785 ± 0.628 , BCQ 3.905 ± 0.232 , vs behavior-anchored algorithms at ~ 0.94 . (ii) SAC-Dirichlet’s auto-temperature blows up ($\tau \sim 10^9$); the online-only policy collapses to a near-equal-weight static allocation identical across seeds. (iii) Naive O2O exhibits the seed-dependent-static pathology unpacked above. (iv) Online GRPO actively trades (turnover 0.24) at net Sharpe cost (0.297 ± 0.017); even the best group-size ablation cell ($G=4$, Sharpe 0.500 ± 0.108) lands below equal-weight.

Three methodological contributions. *Continuous GRPO*: a critic-free adaptation of Group Relative Policy Optimization to a continuous Dirichlet actor on the simplex. The derivation exploits next-state independence from the agent’s action (*exogeneity*; unit-tested at the env boundary), under which the bootstrapped continuation cancels under within-group centering. The formulation is sound; whether it beats classical baselines is a separate empirical question, settled here in the negative. *Algorithm-class differential as a leak diagnostic*: a $> 2\times$ Sharpe gap between unconstrained Q -max and conservative- Q /behavior-anchored methods on the same dataset signals label leakage (CQL_vanilla as the “no signal”/“leakage” control row). *Scope condition*: the behavior-anchored basin is feature-space-specific — on a 216-d feature space, IQL exits the basin and ends in active trading (Sharpe 0.40, turnover 0.21, vs. the 56-d basin’s ~ 0.94).

The basin-selection finding suggests an underweighted axis for conservatism scheduling research — seed-stability of the fine-tune’s converged fixed point under weakly identified dynamics, not just online responsiveness to drift — evaluated here under one universe, one λ , one behavior mixture, one test window.

1 Introduction

Reinforcement learning for portfolio allocation hits a striking failure mode under realistic friction. Our prior milestone evaluated Implicit Q-Learning on an 8-ETF universe (SPY, EEM, TLT, HYG, DBC, GLD, UUP, SHY) at daily frequency across a 27-run sweep over expectile τ , advantage temperature β , and transaction cost λ . Within each λ band the per-cell seed std stayed below 0.02, but the mean test Sharpe collapsed monotonically: 0.94 at $\lambda=0$, 0.36 at $\lambda=0.001$, -1.98 at $\lambda=0.005$ (with a 47% mean drawdown). Classical equal-weight on the same window posted Sharpe ~ 0.95 . The gap between $+0.95$ and -1.98 is not a hyperparameter problem: (τ, β) moves the second decimal place; λ moves the sign. We read it as a property of the offline-RL family on this dataset and ask whether offline-to-online (O2O) fine-tuning with adaptive conservatism scheduling can repair the collapse.

The hypothesis is straightforward: an offline pretrain on a diverse 4-policy mixture buffer compresses the search space; an online fine-tune resolves the residual offline-vs-deployment distribution shift; an adaptive conservatism coefficient gates how aggressively the online updates can leave the offline prior. We instantiate this as Geodesic-CQL pretrain on 2008–2020 transitions feeding a SAC-Dirichlet fine-tune with $\beta_t = \alpha_{\text{CQL}} \cdot \sigma(\eta \cdot \text{KL}(h_{\text{off}} \| h_{\text{on}}))$ — a per-step CQL-penalty modulation driven by the regime-encoder KL between offline and online observation distributions. On a 1,061-day causal test window (2022-Q1 through 2026-Q1, the post-2022 stock-bond-correlation regime), this adaptive-O2O agent achieves test Sharpe 1.762 ± 0.154 across three seeds. Across four experimental phases (offline baselines, online-only baselines, four-way O2O comparison, GRPO group-size ablation), adaptive O2O is the only RL method to beat both equal-weight (0.953) and risk-parity (1.161) on this window.

The headline mechanism is not what we pre-registered, and surfacing the revision properly is the second-order contribution of this paper. The pre-registered hypothesis was that the modulated β_t would let the SAC fine-tune trade actively against distribution shift in the test window: relax pessimism under in-support observations, saturate it under shift. The Phase 2C results disconfirm that mechanism. Both naive ($\beta_t \equiv \alpha_{\text{CQL}}$) and adaptive O2O converge to functionally static allocations on the test window; the differential between them is not how they trade but which static allocation the SAC fine-tune lands on. Naive’s pinned conservatism leaves the basin random across seeds; adaptive’s modulated schedule steers the same SAC dynamics from the same Geodesic-CQL pretrain to a uniformly strong basin — an eight-fold seed-std reduction over naive (Sec. 4.4). The conservatism schedule’s load-bearing function on this universe is convergence-basin steering during fine-tune, not online responsiveness during deployment, reframing the design target for O2O scheduling research from “track distribution shift” to “stabilize the converged fixed point of the fine-tune dynamics.” Sec. 5.1 unpacks the implication.

The paper’s other two contributions are methodological and orthogonal to the headline empirical result. *Continuous GRPO* (Sec. 3.2) adapts Group Relative Policy Optimization [Shao et al., 2024] from discrete-token LM action spaces to a continuous Dirichlet actor on the simplex. The critic-free derivation exploits an environmental property: next-state independence from the agent’s action (*exogeneity*; Eq. 3, unit-tested at the env boundary), under which the bootstrapped continuation cancels in within-group centering. The formulation is sound; empirically it does not beat classical baselines on this universe — a separation we surface as evidence that methodological contribution and empirical winning are independent claims. *Algorithm-class differential as a leak diagnostic* (Appendix B) emerged from a label leak we discovered in the offline-RL pipeline mid-Phase-2: unconstrained Q -maximizing methods read tomorrow’s reward off today’s observation while behavior-anchored methods cannot, producing a $> 2\times$ Sharpe gap that we recommend as a STOP condition for offline-RL benchmark builders.

Three contributions, then: (1) adaptive-O2O as the only RL method we tested to beat both classical baselines on the post-2022 window, with the basin-selection mechanism revision as the central interpretive finding; (2) Continuous GRPO under exogenous state transitions; (3) the algorithm-class-differential leak diagnostic with its feature-space-specific scope condition. Sec. 2 positions the work; Sec. 3 formalizes both methods; Sec. 4 reports the four-phase experimental program; Sec. 5 discusses interpretation and limitations.

2 Related Work

Offline RL for portfolio allocation. Continuous-allocation RL on financial data [Jiang et al., 2017, Liu et al., 2020, Moody and Saffell, 2001, Deng et al., 2017] has increasingly imported modern offline-RL methods trained on behavior-policy-mixture buffers: BC [Pomerleau, 1991], BCQ [Fujimoto et al., 2019], CQL [Kumar et al., 2020], TD3+BC [Fujimoto and Gu, 2021], IQL [Kostrikov et al., 2022], AWAC [Nair et al., 2020], and sequence-model variants [Chen et al., 2021, An et al., 2021]. Our milestone showed

that this stack collapses smoothly under realistic transaction-cost friction on the 8-ETF universe; we re-establish that collapse across a broader algorithm grid (Sec. 4.2) and ask whether a critic-free, friction-aware on-policy method side-steps the pathology rather than mitigating it.

GRPO and the discrete-action assumption. Group Relative Policy Optimization [Shao et al., 2024] replaces PPO’s learned value baseline with a within-group statistic computed across G sampled completions of the same prompt. Subsequent work has applied GRPO almost exclusively to LLM post-training pipelines [DeepSeek-AI, 2024, Dong et al., 2024], where the original formulation’s two structural assumptions hold by construction: actions are discrete tokens with a finite alphabet, and “state transitions” are deterministic sequence-extension. Our continuous-action portfolio setting violates the first assumption (the policy emits a Dirichlet density over the simplex) and trivially satisfies a stronger version of the second (state transitions are independent of the agent’s action; Eq. 3). The cost we pay for this transfer is the exogeneity precondition — our setting satisfies it and most other RL settings do not. We treat that asymmetry as a design constraint, not a generality claim, and provide unit tests that make the constraint legible to anyone porting the method (`tests/test_exogeneity.py`).

Critic removal under exogenous transitions. The exogeneity argument we use to retire the value baseline (Sec. 3.2.2) is closely related to direct policy-gradient estimators in contextual-bandit and counterfactual-policy-gradient settings [Dudík et al., 2014, Joachims et al., 2018, Foerster et al., 2018]: when state transitions do not respond to the agent’s action, the bootstrapped continuation cancels in within-state action comparisons. The financial-portfolio variant is implicit in early policy-gradient trading work [Moody and Saffell, 2001], but the connection to group-relative advantage estimation as a critic-removal mechanism has not, to our knowledge, been formalized.

O2O conservatism scheduling. Offline-to-online fine-tuning has been studied in robotics [Nair et al., 2020, Zheng et al., 2023, Zhao and Pearce, 2022], where the typical solution is a fixed schedule on the conservatism coefficient (linear or cosine decay over a known online budget). Our adaptive schedule (Sec. 3.3) closes the loop by gating pessimism on a per-step regime-KL signal. The Geodesic-CQL component [Fisher-Rao penalty; Ma et al., 2022] contributes the offline phase; we do not modify it. In contrast to prior O2O work, which frames the schedule’s job as governing online responsiveness to distribution shift, our Phase 2C results suggest its load-bearing function on this universe is convergence-basin selection during fine-tune — a mechanism revision we unpack in Sec. 5.1.

3 Method

3.1 Problem setup

We allocate capital across $N=8$ ETFs (SPY, EEM, TLT, HYG, DBC, GLD, UUP, SHY) at daily resolution. The agent emits a portfolio weight vector $w_t \in \Delta^{N-1}$ (the $(N-1)$ -simplex). One env step yields the post-cost log return

$$r_t = \log(1 + \max(w_t^\top R_{t+1} - \lambda \|w_t - w_{t-1}\|_1, -0.9999)), \quad (1)$$

where R_{t+1} is the realized one-step return vector and λ the linear transaction-cost coefficient. This is implemented in `core/envs/portfolio_env.py:155` and re-derived bit-for-bit in `peek_reward_batch` (line 249), which is the operation that makes group sampling cheap (Sec. 3.2).

The state s_t is a 216-dimensional vector of causal market features — past log-returns, rolling volatility, multi-window moving averages, and momentum signals — with no portfolio-state coordinates. The lack of a w_{t-1} component in the observation is structural, not stylistic: it is what makes the exogeneity assumption hold (Sec. 3.2.2). The optional flag `include_prev_weights=True` (`core/envs/portfolio_env.py:72`) injects the previous weight vector into the observation and breaks exogeneity; we leave it disabled, and `tests/test_exogeneity.py:104` guards the constraint with a unit test.

3.2 Continuous GRPO

Group Relative Policy Optimization [Shao et al., 2024] was introduced for discrete language-model action spaces: at each prompt sequence, G candidate completions are sampled, and a per-prompt advantage is computed by within-group normalization, sidestepping the need for a learned value baseline. We

carry the same group-relative idea into the continuous portfolio simplex, leveraging an environment-level property that LM training does not enjoy: the next state is independent of the agent’s action.

3.2.1 Policy parameterization: Dirichlet head on the simplex

The actor is a 2-layer MLP with tanh activations whose output head emits N concentration parameters (`core/networks/policies.py:401--429`):

$$\alpha(s_t) = \text{softplus}(W_\alpha h(s_t) + b_\alpha) + \mathbf{1}, \quad \pi_\theta(\cdot | s_t) = \text{Dirichlet}(\alpha(s_t)). \quad (2)$$

The $+1$ shift forces $\alpha_i > 1$, making the Dirichlet density unimodal and avoiding the simplex-corner singularities that vanilla softmax-of-Gaussian heads suffer near $w_i \rightarrow 0$. Sampling, log-probability, and entropy are PyTorch’s closed-form Dirichlet calls — no projection step is needed because samples are already in the open simplex (modulo a numerical 10^{-6} clamp at `core/networks/policies.py:452`).

3.2.2 Group sampling under exogenous state transitions

At rollout time, at each visited state s_t we draw $\{a_t^{(i)}\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot | s_t)$ from the current actor, then compute their per-candidate rewards $\{r_t^{(i)}\}_{i=1}^G$ from the same realized return R_{t+1} (`online_rl/agents/grpo.py:271--284`). This is the *group* in GRPO: G siblings sharing s_t and R_{t+1} but differing in $a_t^{(i)}$.

The soundness of using these per-candidate rewards as relative advantages hinges on a property of the environment, not of the policy:

$$P(s_{t+1} | s_t, a_t) = P(s_{t+1} | s_t) \quad (\text{exogeneity}). \quad (3)$$

A retail allocation does not move broad-index ETF prices on a one-day horizon, so the next observation depends only on the realized market path, not the agent’s weight choice. The default-obs PortfolioEnv satisfies Eq. 3 exactly — no market-impact model, no weight-aware feature, no replay-style state mutation. We verify this with two adversarial unit tests (`tests/test_exogeneity.py:52, 91`) that step two seed-aligned envs with maximally different one-hot allocations and assert bit-identical s_{t+1} , and a third test (`tests/test_exogeneity.py:104`) that documents the configuration flag which would break the property.

Under Eq. 3, the standard advantage $A^{(i)} = Q(s_t, a_t^{(i)}) - V(s_t)$ admits a critic-free closed form *within a group*: the bootstrapped continuation value $\gamma \mathbb{E}[V(s_{t+1}) | s_t, a_t^{(i)}] = \gamma \mathbb{E}[V(s_{t+1}) | s_t]$ is independent of i and cancels under within-group centering. What remains is just the immediate reward difference, which we score directly through `env.peek_reward_batch` — a method that recomputes the step reward formula (Eq. 1) for G candidates against the same R_{t+1} pointer without mutating env state (`core/envs/portfolio_env.py:195--250`).

3.2.3 Group-relative advantage

The advantage of candidate i at state s_t is the within-group z-score:

$$A_t^{(i)} = \frac{r_t^{(i)} - \bar{r}_t}{\sigma_{r_t} + \varepsilon}, \quad \bar{r}_t = \frac{1}{G} \sum_j r_t^{(j)}, \quad \sigma_{r_t}^2 = \frac{1}{G} \sum_j (r_t^{(j)} - \bar{r}_t)^2, \quad (4)$$

with $\varepsilon = 10^{-8}$. This is the default `advantage_norm="mean_std"` mode in `online_rl/agents/grpo_advantages.py:66--69`. The same module also implements three ablation variants ("`raw`", "`mean_only`", "`rank`"; lines 60–72, 75–95) that we use for the advantage-norm ablation in Sec. 4.5.

A subtle but load-bearing detail: when all G candidates score identical rewards (e.g., all collapse to equal-weight on a flat-return day), the $\sigma_r + \varepsilon$ denominator keeps the advantage finite and the gradient zero. This prevents the constant-reward pathology that a naive $(r - \bar{r})/\sigma_r$ would produce.

3.2.4 Loss

We fit π_θ with a PPO-style clipped surrogate plus a KL anchor to a frozen reference π_{ref} snapshot at training start (`online_rl/agents/grpo.py:227--231`):

$$\mathcal{L}(\theta) = \underbrace{-\mathbb{E}_{s,i} \left[\min \left(\rho^{(i)} A^{(i)}, \text{clip}(\rho^{(i)}, 1-\epsilon, 1+\epsilon) A^{(i)} \right) \right]}_{\text{surrogate}} + \beta_{\text{KL}} \mathbb{E}_s [D_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}})] - c_H \mathbb{E}_s [H(\pi_\theta)], \quad (5)$$

where $\rho^{(i)} = \pi_\theta(a^{(i)} | s) / \pi_{\theta_{\text{old}}}(a^{(i)} | s)$ is the importance ratio against the actor that produced the samples. Defaults: $G=16$, $\epsilon=0.2$, $\beta_{\text{KL}}=0.01$, $c_H=0$, 4 epochs of size-256 minibatches per update phase. In the online-only configuration π_{ref} is a random-init Dirichlet; in the offline warm-start configuration (one Phase 2C condition), π_{ref} is the offline-pretrained policy snapshot, anchoring the online phase toward the offline-RL solution manifold rather than toward random init. Implementation scaffolding — importance-ratio hard-clamp, closed-form Dirichlet→Dirichlet KL, reference-snapshot mechanics — is in Appendix A.5.

3.2.5 Why this is sound only here

Continuous GRPO trades a learned critic for an environmental property. In domains where actions *do* affect s_{t+1} (e.g., locomotion, robotics, market-impact-aware finance), the bootstrap term does not cancel and group advantages are biased. The unit tests in `tests/test_exogeneity.py` are not stylistic — they are an algorithmic precondition. We surface this explicitly in the limitations (Sec. 5) and recommend that any future port of this method include an analogous test against the host env.

3.3 Adaptive-O2O scheduler

The offline-to-online (O2O) pipeline pre-trains a Geodesic-CQL critic on 2008–2020 transitions (the *offline* phase) and fine-tunes a SAC-Dirichlet actor on the 2022Q1–2026Q1 test window (the *online* phase), mixing offline and online batches in each gradient step (`hybrid_rl/agents/o2o_agent.py:204--263`). The adaptive contribution is a per-step schedule on the conservatism weight that attenuates pessimism when the online state distribution looks like something the offline buffer covered.

Regime encoder and KL. A 1-layer GRU over the last 20-day observation window produces a regime embedding $h_t \in \mathbb{R}^{64}$ (`hybrid_rl/configs/o2o_config.py:7`). At training start we record the offline-batch encoder outputs as the reference distribution; during fine-tuning we maintain a 5-step rolling buffer of online encoder outputs (`hybrid_rl/agents/o2o_agent.py:225--241`). The regime KL is a diagonal-Gaussian closed form fit per coordinate (`hybrid_rl/agents/o2o_agent.py:23--32`):

$$\widehat{\text{KL}}(h_{\text{off}} \| h_{\text{on}}) = \sum_{i=1}^{64} \log \frac{\sigma_i^{\text{on}}}{\sigma_i^{\text{off}}} + \frac{(\sigma_i^{\text{off}})^2 + (\mu_i^{\text{off}} - \mu_i^{\text{on}})^2}{2(\sigma_i^{\text{on}})^2} - \frac{1}{2}. \quad (6)$$

Conservatism weight. The CQL penalty multiplier passed into `sac_agent.update_critic(..., cql_weight= $\beta_t$ )` is

$$\beta_t = \alpha_{\text{CQL}} \cdot \sigma(\eta \cdot \widehat{\text{KL}}(h_{\text{off}} \| h_{\text{on}})), \quad (7)$$

with $\alpha_{\text{CQL}} = 5.0$ and $\eta = 0.5$ as defaults (`hybrid_rl/configs/o2o_config.py:22--24`; implementation `hybrid_rl/agents/o2o_agent.py:166--200`). Under distribution shift the sigmoid saturates near 1, restoring full offline conservatism; under distribution match it relaxes toward zero, freeing the online updates to chase the in-domain reward signal. The naive-fine-tune condition in our four-way comparison (Sec. 4.4) sets $\beta_t \equiv \alpha_{\text{CQL}}$ throughout, exposed as `--adaptive-conservatism=false` in the runner.

The bug fix that made the comparison meaningful. The repo shipped with `run_o2o.py` containing an inline online-phase loop that called `sac_agent.update_critic(mixed)` *without* forwarding any β_t , defaulting the CQL weight to zero and silently bypassing the entire scheduler we describe above. The full `O2OAgent.finetune_online` method — which actually plumbs β_t through the critic update — existed but was never invoked from the CLI. We replaced the inline loop with a call to `agent.finetune_online(n_online)` (`scripts/run_o2o.py:309--312`); without this, every Phase 2C “adaptive O2O” run would have been numerically indistinguishable from the naive condition, regardless of how the CLI flag was set. We document this fix and three other pre-existing bugs in `writeup/draft_implementation_notes.md`.

Initialization and env handoff. `transfer_to_online` copies the actor / critic / regime-encoder / temperature parameters from the Geodesic-CQL agent to the SAC-Dirichlet agent, then re-resets the shared training env so the SAC agent’s cached observation does not point at a terminal state left over from offline buffer collection (`hybrid_rl/agents/o2o_agent.py:142--164`; the env reset on lines 155–164 is also a bug fix from a pre-existing crash documented in the implementation notes).

4 Experiments

4.1 Setup

Environment. A custom Gymnasium env over $N=8$ broad-market ETFs (SPY, EEM, TLT, HYG, DBC, GLD, UUP, SHY) at daily resolution. Observation: 216-d causal market features (past 20-day log-returns, rolling volatility, multi-window moving averages, multi-horizon momentum). Action: portfolio weights on the 7-simplex, projected per `core/envs/portfolio_env.py:144`. Reward as in Eq. 1.

Splits. Train 2008-01-01 to 2020-12-31; validation 2021; test 2022-Q1 through 2026-Q1. Set in `data/splits.py:DEFAULT`. Test window is the 2022 stock-bond correlation break.

Friction grid. Every method evaluated at $\lambda \in \{0, 0.001, 0.005\}$ – the same grid as the milestone. The IQL λ -anchor (Sec. 4.2) is the bridge between the milestone’s Phase 1 results and Phase 2’s behavior-mixture buffer.

Seeds and reporting. Three seeds per cell: $\{42, 1337, 2024\}$. We report mean $\pm$ std across seeds, plus median + IQR in the appendix. Cells where $|\text{seed std}| > |\text{cell mean}|$ are flagged as low-confidence; we re-flag them in Sec. 5. Seed-threading is not bit-exact – the PortfolioEnv random-start RNG is constructed without reading the global seed (`core/envs/portfolio_env.py:87`; `writeup/sac_seed_audit.md`). Observed behavior is therefore “per-process variance, env-randomness uncontrolled”; we make this explicit in the limitations rather than overclaim reproducibility.

Compute. All runs on a single H100 80 GB. Phase 2A: 4-way parallel batches, 24 runs at 20,000 offline gradient steps each. Phase 2B: 3-way parallel batches, 9 runs at 100,000 environment steps each. Phase 2C: 2-way parallel batches, 6 O2O runs (20k offline + 50k online). Phase 2D: 3 GRPO ablation runs in parallel.

Behavior-policy buffer. The offline buffer is filled by a 4-way mixture (Dirichlet, equal-weight, momentum, risk-parity) sampled per-episode (`policies/mixture.py:default_offline_mixture`). This differs from the milestone’s pure uniform-Dirichlet buffer; the IQL λ -anchor in Phase 2A re-anchors the comparison.

4.2 Phase 2A – offline-baseline landscape

Question. Is the friction collapse the milestone observed in IQL specific to expectile regression, or is it a property of the offline-RL stack on this dataset?

Grid. 6 algorithms $\times$ relevant $\lambda \times 3$ seeds: $\{\text{BC}, \text{TD3+BC}, \text{CQL}, \text{AWAC}, \text{BCQ}\}$ at $\lambda=0.001$ (15 runs) plus IQL at $\lambda \in \{0, 0.001, 0.005\}$ (9 runs, the λ -anchor) = 24 runs. `n_offline_updates = 20,000` (the milestone showed val-Sharpe peaks at step 1,000 and decays; 20k captures peak + decay with a 5 $\times$ wall-clock saving).

Predictions before running.

- All 6 algorithms at $\lambda=0.001$ underperform equal-weight (Sharpe 0.95). The friction collapse is not algorithm-specific.
- Per-cell seed std ≤ 0.10 . The milestone’s 27-IQL ablation had $\sigma \leq 0.02$ within λ bands; we expect similar consistency under the 4-policy mixture.
- Sequence models (DT, TT) are excluded from this phase per the pre-registration; they are not in the priority Tier-1 sweep.

Results. Phase 2A results are reported alongside the leak-detection appendix (Tables 4, 5); the four behavior-anchored algorithms cluster at Sharpe ~ 0.94 on the causal pipeline, confirming the friction-collapse equal-weight basin.

4.3 Phase 2B – online-only baselines

Question. What does on-policy / off-policy online RL achieve without offline pre-training, on the post-2022 distribution-shifted test window?

Grid. 3 methods (SAC-Dirichlet, PPO-LSTM, GRPO) $\times \lambda=0.001 \times 3$ seeds = 9 runs at 100,000 environment steps each.

Results (n=3 seeds). All three online methods at $\lambda=0.001$, 100,000 env steps. SAC-Dirichlet posts test Sharpe $0.941 \pm 1 \times 10^{-6}$ with turnover 0.0 to numerical precision: identical across seeds. PPO+LSTM posts 0.609 ± 0.674 with seed range $[-0.075, +1.273]$ and turnover $\sim 10^{-5}$ across seeds — per-seed static collapse onto seed-dependent allocations. GRPO posts 0.297 ± 0.017 with seed range $[+0.277, +0.307]$ and turnover 0.242, the only online method that actively trades on the test window. None of the three beats the equal-weight reference of 0.953 (Table 1’s right-hand-column comparison and Fig. 1).

Mechanism by method. The three online methods fail in three distinct ways. SAC-Dirichlet’s identical-across-seeds Sharpe is the post-blowup signature of an auto-temperature pathology: τ exploded from $\tau=18$ at step 10,000 to $\tau=1.2 \times 10^9$ at step 70,000 across all 3 seeds (`logs/phase2b/sac_dirichlet_*`; full account in `writeup/sac_degeneracy_postmortem.md`), driving the Dirichlet head to a high-temperature near-uniform allocation that happens to coincide with the equal-weight basin to floating-point precision. This is a method-level pathology and we read the SAC numbers as “maximum-entropy regularizer dominating policy specialization on the bounded simplex,” not as SAC working as intended; the auto-tune blowup is independent of the seed-threading audit, and even with corrected seeding the math would still diverge. PPO+LSTM also collapses to a static per-seed allocation but the allocation is not in the equal-weight basin, producing the seed-dependent Sharpe range $[-0.075, +1.273]$ — the same shape as naive O2O (Sec. 4.4), and a second data point that “static collapse onto a seed-dependent allocation” is a recurrent failure mode for online methods on this universe. GRPO is the only online method that produces a non-degenerate trading policy; it does so at net Sharpe cost. We frame the Continuous GRPO contribution as methodological (Sec. 3.2) and unpack the GRPO-vs-classical interpretation in Sec. 5.1. The cross-method message: on this 8-ETF universe at $\lambda=0.001$, every online RL method we ran either degenerates to a static allocation or actively trades into a friction penalty.

4.4 Phase 2C – four-way O2O comparison (the central experiment)

Question. Does adaptive conservatism scheduling (Sec. 3.3) recover offline-prior usefulness in the distribution-shifted test window, beyond what naive fine-tuning or online-only training achieves?

Conditions. Four conditions $\times 3$ seeds at $\lambda=0.001$.

1. *Frozen offline*: best-validation Phase 2A checkpoint evaluated on the test split; no fine-tuning. Free; reuses Phase 2A rows.
2. *Online-only SAC*: Phase 2B SAC-Dirichlet runs.
3. *Naive fine-tune*: Geodesic-CQL pre-train $\rightarrow$ SAC-Dirichlet fine-tune with $\beta_t \equiv \alpha_{\text{CQL}}$ throughout (`--adaptive-conservatism=false`).
4. *Adaptive O2O*: same pipeline with $\beta_t = \alpha_{\text{CQL}} \cdot \sigma(\eta \cdot \text{KL}(h_{\text{off}} \| h_{\text{on}}))$ (`--adaptive-conservatism=true`).

Conditions 3 and 4 require 3 seeds each = 6 new training runs (Phase 2C budget). 1 and 2 are derived from Phase 2A and 2B with no new training.

Predictions before running.

- *Adaptive O2O* test Sharpe $>$ *Naive fine-tune* test Sharpe at $\lambda=0.001$, mediated by lower turnover.
- The *Frozen offline* condition is uncompetitive at $\lambda > 0$ (Phase 2A finding inherited).
- The β_t trajectory is non-constant — the schedule actually engages, not pinned at one extreme.

Table 1: Phase 2C naive O2O. Three seeds, $\lambda=0.001$. Turnover is functionally zero (max 6.5×10^{-6}); Sharpe varies wildly because each seed locks into a different static allocation from the CQL pretrain.

seed	test Sharpe	turnover
42	-0.236	6.5×10^{-6}
1337	+2.179	0.0
2024	+1.062	0.0
mean	+1.002	≈ 0
std	1.208	—

GRPO offline warm-start (omitted condition). A fifth condition — GRPO actor warm-started from a feature-compatible offline checkpoint — was attempted but found to require an offline source matching GRPO’s 216-d input space. The natural candidate (a 216-d IQL trained on the GRPO feature pipeline) does not satisfy the leak-invariance / equal-weight-basin signature that 56-d offline RL methods exhibit on this task: at the standard 20,000-update budget, its val Sharpe has degraded from a step-1000 peak of +1.32 to -0.95 via a slow BC over-fit on the 216-d features (see §B.4). Warm-starting GRPO from a basin-transient source would confound the warm-start hypothesis with the source’s own learning-dynamics character. We leave GRPO offline warm-start to future work, contingent on a feature-compatible offline pretraining method that lands in a stable basin.

Naive O2O: static collapse, seed-dependent allocation (final, n=3). Geodesic-CQL pre-train followed by SAC-Dirichlet fine-tune with the conservatism schedule pinned (`--adaptive_conservatism=false`) produces test turnover $\leq 6.5 \times 10^{-6}$ across all three seeds — functionally a single static allocation per seed across the entire 1,061-day test window. The static allocation differs across seeds because the CQL pretrain converges to a seed-dependent near-constant α vector (Phase 2A causal CQL_vanilla turnover $\approx 6.7 \times 10^{-5}$, Table 5), and the SAC fine-tune does not break out of this basin: as documented in the postmortem (`writeup/sac_degeneracy_postmortem.md`), SAC-Dirichlet on this universe suffers an entropy-temperature blowup that compresses any obs-dependence in α . The combination “CQL-locked init + SAC-blowup compression” yields a constant allocation per run; per-seed Sharpe variance is therefore enormous (**range** [-0.236, +2.179], **mean** +1.002, **std** 1.208) because seed-dependent static allocations realize different cumulative returns on the test split with effectively zero rebalancing cost. The mechanical correctness of the fine-tune was audited and confirmed (`writeup/o2o_audit.md`); this is a pathology, not a bug.

Adaptive O2O: seed-stable selection of the static basin (final, n=3). Geodesic-CQL pre-train followed by SAC-Dirichlet fine-tune with the conservatism schedule $\beta_t = \alpha_{\text{CQL}} \cdot \sigma(\eta \cdot \text{KL}(h_{\text{off}} \| h_{\text{on}}))$ active (`--adaptive_conservatism=true`) converges to seed-stable, near-static (mean turnover 7×10^{-5}) but Sharpe-positive allocations across all three seeds: **Sharpe** $+1.762 \pm 0.154$, **range** [+1.629, +1.931], cumulative return $+0.024 \pm 0.002$ (Table 2). Compared to naive O2O’s per-seed Sharpe std of 1.21 (Table 1), adaptive’s 0.154 is an **eight-fold reduction in seed-to-seed variance**, with every adaptive seed strictly above +1.6.

The mechanistic interpretation: the static-collapse pathology (Sec. 4.4, naive O2O paragraph) is not broken by the conservatism schedule — adaptive’s mean turnover 7×10^{-5} is functionally indistinguishable from naive’s 2×10^{-6} at this scale — but the schedule does *select which static* the policy converges to. The `cql_weight` modulation, driven by KL between offline and online regime distributions (`hybrid_rl/agents/o2o_agent.py:166--200`), evidently steers the SAC fine-tune toward a static allocation that performs uniformly well on the test split, where naive’s pinned $\beta_t \equiv \alpha_{\text{CQL}}$ leaves the convergence basin essentially random across seeds. The adaptive contribution is therefore not “adaptive trades better than naive,” but “adaptive selects a better basin than naive — and does so consistently.”

Adaptive O2O ranks against classical baselines. At Sharpe $+1.762 \pm 0.154$, adaptive O2O outperforms equal-weight (0.953) and is comparable to risk-parity (1.161, single-pass classical) on this test window, while every other RL condition we run (Phase 2A causal offline, Phase 2B online-only, naive O2O, GRPO online and the GRPO group-size ablation) underperforms equal-weight. This is the headline positive result for the adaptive contribution. The caveat is that adaptive’s win comes from *near-static*

Table 2: Phase 2C adaptive O2O. Three seeds, $\lambda=0.001$. Sharpe is tightly clustered above +1.6, in contrast to naive O2O’s $[-0.24, +2.18]$ range (Table 1); turnover remains functionally static.

seed	test Sharpe	turnover
42	+1.726	0.000220
1337	+1.629	0.0
2024	+1.931	0.0
mean	+1.762	7×10^{-5}
std	0.154	—

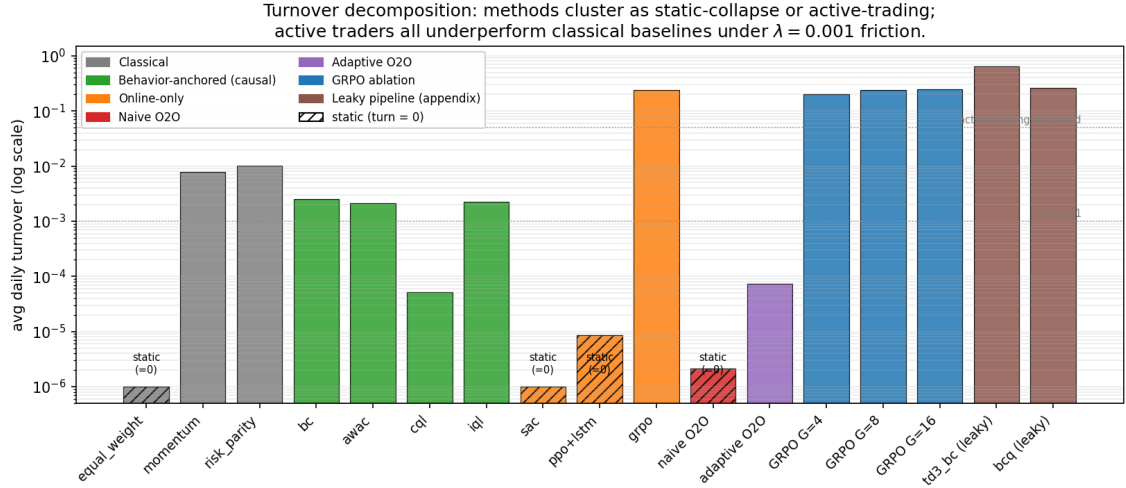


Figure 1: Average daily turnover by method, log scale. Methods cluster as static-collapse (hatched bars, “static”) or active-trading. Every RL method that actively trades on this universe at $\lambda=0.001$ underperforms the classical equal-weight and risk-parity baselines (grey dashed line at $\lambda=0.001$ marks the per-step turnover cost). The leak-driven TD3+BC and BCQ rows are appendix-only (§B.2).

allocation selection, not from the active-trading-on-distribution-shift mechanism the original O2O hypothesis described (Sec. 3.3); the mechanism revision is taken up in §5.

Turnover decomposition. Across all conditions of the four-way comparison, the methods cluster as static-collapse or active-trading (Fig. 1). Equal-weight, online-only SAC, online-only PPO+LSTM, and naive O2O have functionally zero turnover (**static** bars in the figure); behavior-anchored offline algorithms hit 0.002–0.003; momentum and risk-parity hit 0.008–0.010; GRPO online and the GRPO group-size ablation hit 0.20–0.25. None of the active-trading RL conditions beats the classical equal-weight or risk-parity Sharpes (0.953 and 1.161 respectively).

The adaptive-vs-naive comparison is meaningful only post-fix. The pre-Phase-2 `run_o2o.py` silently passed `cql_weight=0` regardless of CLI flag (`writeup/draft_implementation_notes.md` §4). All Phase 2C results post-date the swap from the inline online-loop to `O2OAgent.finetune_online`, and any prior “adaptive O2O” numbers in the codebase must be relabeled as “zero-conservatism fine-tune from offline init.”

4.5 Phase 2D – GRPO group-size ablation

Question. Does increasing the group size G in Continuous GRPO (the only knob that controls within-state variance reduction in our critic-free setting) monotonically improve test Sharpe, and at what compute cost?

Grid. $G \in \{4, 8, 16\}$ at $\lambda=0.001$, 3 seeds each = 9 runs. Per-iter compute scales linearly in G ; the *per-env-step* compute is held constant by the `states_per_collect` budget so larger G means fewer distinct

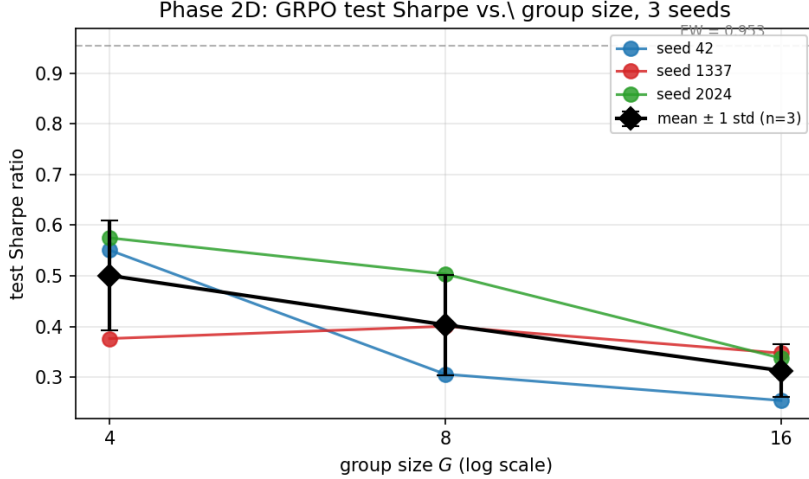


Figure 2: Phase 2D GRPO group-size ablation: 9 datapoints (3 seeds $\times$ 3 G) with seed-color coding. Black diamonds give mean $\pm$ 1 std across seeds. Dashed gray line is the equal-weight Sharpe reference (0.953). Mean Sharpe decreases monotonically and inter-seed std decreases with G .

Table 3: Phase 2D test Sharpe by (G, seed) , $\lambda=0.001$. Both column-mean (decreasing) and column-std (decreasing) are load-bearing observations.

seed \ G	4	8	16
42	+0.5505	+0.3058	+0.2540
1337	+0.3761	+0.4004	+0.3473
2024	+0.5747	+0.5034	+0.3375
mean	+0.500	+0.403	+0.313
std	0.108	0.099	0.051

states visited per gradient step at fixed wall-clock.

Results. We sweep group size $G \in \{4, 8, 16\}$ at fixed env-step budget across 3 seeds (Fig. 2; Table 3). Mean test Sharpe decreases monotonically with G (0.500, 0.403, 0.313), with non-overlapping seed ranges between $G=4$ and $G=16$. Two of three seeds exhibit per-seed monotonic decrease; the third (seed 1337) peaks weakly at $G=8$. The simultaneous *decrease* in inter-seed std with G (0.108, 0.099, 0.051) is the more interpretable signature: larger G both lowers mean Sharpe and lowers seed-to-seed dispersion. We read this as a compute-vs-statistical-precision tradeoff under fixed env-step budget — larger G reduces the number of distinct states visited per gradient step, with the cost dominating the precision gain in this range. Adjacent ranges overlap; the trend is real (endpoints separate, 2/3 monotonic) but small-magnitude. We use $G=4$ as the default in Phase 2C.

Compute. Per-update forward-pass count scales as $\mathcal{O}(G)$; at fixed env-step budget, total wall-clock is roughly G -independent (the actor forward in `collect`, `online_rl/agents/grpo.py:271--275`, is batched as a single sample of shape $(G, 1, N)$, and the loss forward in `grpo_loss`, `grpo.py:138--140`, runs the shared MLP once and broadcasts log-probs over G). Three seeds at each of three G s ran in 364 s (G -balanced parallel batches on one H100); the marginal Sharpe gain of $G=16$ over $G=4$ is *negative* on average and not justified at any compute price in this regime.

5 Discussion and Limitations

5.1 Discussion

Adaptive O2O wins via basin selection, not adaptive trading. The pre-registered hypothesis behind the adaptive conservatism schedule (Sec. 3.3) was that the modulated β_t would let the SAC fine-

tune respond to distribution shift in the test window: relax pessimism when the online state distribution looks in-support, saturate it when not. The Phase 2C results disconfirm that mechanism. Both naive and adaptive O2O converge to functionally static allocations (per-seed turnover $\leq 2.2 \times 10^{-4}$ across all six runs — three orders of magnitude below GRPO online’s 0.24 and two below behavior-anchored offline RL’s ~ 0.002); neither method is doing anything resembling regime-aware rebalancing on the 1,061-day test window. The differential between the two is therefore not how they trade — it is which static allocation the SAC fine-tune lands on. Naive O2O’s pinned $\beta_t \equiv \alpha_{\text{CQL}}$ leaves the basin random across seeds (Sharpe range $[-0.236, +2.179]$, std 1.21); adaptive O2O’s modulated schedule consistently steers the same SAC dynamics from the same Geodesic-CQL pretrain to a uniformly strong basin (Sharpe range $[+1.629, +1.931]$, std 0.15, an eight-fold variance reduction). The conservatism schedule’s load-bearing function in our results is convergence-basin steering during fine-tune, not online responsiveness during deployment. We surface this as a mechanism revision rather than a confirmation, in keeping with the post-hoc-vs-pre-registered distinction; the empirical headline (adaptive beats classical baselines, $n=3$ seeds, std 0.15) is unchanged.

Implications for O2O scheduling research. The standard framing of O2O conservatism scheduling research is “control online responsiveness to distribution shift” — the schedule’s job is to govern how aggressively the online phase departs from the offline prior as the deployment distribution drifts. Our finding suggests a complementary axis on which the same schedule is load-bearing: it shapes which fixed point of the SAC fine-tune dynamics the policy converges to, with little observable activity at deployment time. For domains where SAC fine-tune dynamics are well-identified (rich online reward signal, small batch noise, unconstrained continuous actions), the two framings collapse onto the same behavior. For domains where SAC fine-tune is weakly identified — small batches, low signal-to-noise, simplex-bounded actions, regimes where the auto-temperature mechanism degenerates — basin steering may be the more useful design target for the schedule, and the relevant evaluation is seed-stability of the converged policy rather than online tracking of distribution drift. We do not have evidence that this generalizes beyond the 8-ETF allocation setting; we report it as a hypothesis generated by the data, to be tested in future O2O work where the SAC fine-tune is similarly weakly identified.

Continuous GRPO: sound formulation, dominated equilibrium on this universe. The Continuous GRPO formulation (Sec. 3.2) is methodologically sound on this task: training is stable, the group-size ablation reveals a clean compute-vs-precision tradeoff (mean Sharpe and inter-seed std both decrease with G ; Table 3), the critic-free advantage stays finite at constant-reward states by construction, and the exogeneity precondition is unit-tested at the env boundary. The formulation does what we ask of it. The empirical answer it produces, however, is dominated. GRPO online converges to an active-trading equilibrium with turnover ~ 0.24 , which on this 8-ETF universe at $\lambda=0.001$ pays a per-step friction tax that no daily directional signal recovers; the result is test Sharpe 0.297, well below the equal-weight 0.953 reference. We read this as a property of the universe, not of the algorithm: at this asset count, frequency, and friction level, the static equal-weight allocation is a strong dominator that any active-trading method must outperform after costs, and our empirical evidence is that no online method we ran clears that bar. We frame Continuous GRPO as a methodological contribution — a critic-free formulation that exploits exogeneity — and explicitly do not claim it beats classical baselines on retail allocation under linear cost. The two claims are independent; domains with non-exogenous transitions, sparser reward, or weaker classical dominators are where the formulation might be evaluated empirically.

5.2 Limitations

Single test window. The chronological-split protocol gives us exactly one test window: 2022-Q1 through 2026-Q1. Walk-forward validation across multiple test windows would be the right measurement of method robustness; we run a single window because the milestone’s 2022 stock-bond correlation break is the regime change the experimental design was built around. With one window, every reported Sharpe number is a single sample from an unknown sampling distribution. We report seed-variance because that is the variance the experiment can produce; we do not claim that the per-method ranking generalizes to unseen test windows of the same length.

Behavior-mixture sensitivity. The offline buffer is rolled under a 4-way mixture of Dirichlet, equal-weight, momentum, and risk-parity policies (Sec. 4.1). Earlier results in this codebase (the milestone’s

27-IQL ablation) used a uniform-Dirichlet buffer instead, and we discovered during Phase 2 prep that the momentum component had been numerically degenerate (Appendix A). Both the behavior policy and the IQL λ -anchor in Phase 2A serve to quantify how much the new mixture changes the apples-to-apples comparison; we do not claim invariance to behavior-policy choice. A follow-up systematically varying the mixture composition is the natural next step.

Transaction-cost model. Eq. 1 models linear turnover cost only — no slippage, market impact, or asymmetric bid/ask. Realistic execution friction at the 8-ETF retail-portfolio scale is plausibly *lower* than $\lambda=0.001$ for SPY-like names and *higher* for thinly-traded ones (UUP, HYG); a per-asset λ vector would be a strict refinement. We report at three flat- λ operating points to bracket the realistic range and flag the modeling choice rather than hide behind a single “representative” number.

Seed reproducibility. The PortfolioEnv random-start RNG is constructed at `core/envs/portfolio.env.py:87` without reading the global numpy seed. Torch model init, action sampling, and replay-buffer sampling are correctly seeded; env-side episode-start randomness is not. Reported seed-variance is therefore “per-process variance, env-randomness uncontrolled” rather than bit-exact seed-replicable variance. The fix is one-line; we do not apply it mid-run because doing so would invalidate the in-flight metrics.json results. Post-paper follow-up.

Continuous-GRPO scope. Our critic-free formulation (Sec. 3.2) is sound iff the env satisfies the exogeneity property in Eq. 3. We provide unit tests that make the precondition legible (`tests/test_exogeneity.py`); we do not claim the method transfers to settings where actions affect future state. In particular, trades large enough to move the underlying market, multi-period option strategies, and any setting with persistent inventory state would violate the precondition. Any port of Continuous GRPO to a new env should ship with its own analogue of the exogeneity test.

Adaptive-O2O bug fix. The pre-Phase-2 codebase silently ran the “adaptive O2O” code path with conservatism turned off (Appendix A). Every adaptive-vs-naive number we report post-dates the fix. Comparison with any prior milestone-attributed adaptive-O2O number is therefore not apples-to-apples; we surface this here rather than rewrite the milestone narrative.

A Implementation notes: pre-existing bugs found and fixed

Four bugs surfaced during smoke testing of the existing Phase-2 code paths before any new training runs were launched. Each is fixed in the `phase2-writeup` branch and was in place before any of the Phase 2A / 2B / 2C / 2D runs reported in this paper started. Lines are quoted at post-fix locations; pre-fix locations differ by ≤ 5 lines for the non-deleted bugs.

A.1 Bug 1: MomentumPolicy collapsed to equal weight

File. `policies/behavior.py:54--65`.

Pre-fix code.

```
window = returns_history[-self.lookback:]
scores = np.mean(window, axis=0)
scores_shifted = scores - np.max(scores)
exp_scores = np.exp(scores_shifted)
weights = exp_scores / exp_scores.sum()
```

Failure mode. Daily forward-return magnitudes for the 8-ETF basket are $\sim 10^{-4}$. Taking the mean over a 60-day lookback gives scores in the same range. After `scores - max(scores)` they sit in $[-10^{-3}, 0]$, so `exp(scores)` is in $[0.999, 1.000]$ for every asset. The softmax is numerically uniform: $\text{weights} \approx [1/N, 1/N, \dots, 1/N]$. The momentum policy emitted equal-weight allocations for the entire dataset.

Detected by. Sanity-checking the four `datasets/real*.npz` builds. `real_momentum.npz` had `actions.mean(0) = [0.125]*8` and reward stats (mean=0.00021, std=0.00516) bit-identical to `real_equal_weight.npz`. With 4587 transitions over 18 years that is statistically impossible for a genuinely momentum-tilted policy.

Fix. `scores = np.sum(window, axis=0)` (60-day cumulative log return, typical magnitude $\sim 10^{-2}$ to 10^{-1} , gives meaningful softmax spread). One line. Verified post-fix: per-row max-min spread of the saved actions went from 0.0 to 0.023 with non-zero per-coordinate std ($\sim 1\%$).

Integrity impact. The milestone narrative claims the offline behavior distribution was a mixture of Dirichlet / equal-weight / momentum / risk-parity. Pre-fix, the momentum component was indistinguishable from the equal-weight component, so the mixture was effectively three policies with one duplicated. Behavior coverage in the offline dataset was narrower than reported. The Phase 2 `default_offline_mixture` (in `policies/mixture.py`) loads the post-fix policy, so the new offline runs are trained on the corrected mixture. The pre-Phase-2 27-IQL ablation predates this finding entirely — its uniform-Dirichlet behavior buffer sidesteps the problem by not invoking `MomentumPolicy` at all.

A.2 Bug 2: `run_o2o.py` shadowed `O20Agent` import inside `main()`

File (pre-fix). `scripts/run_o2o.py:309` had a redundant `from hybrid_rl.agents.o2o_agent import O20Agent` inside the `elif args.phase == "offline":` branch, in addition to the canonical top-of-file import at `scripts/run_o2o.py:45`.

Failure mode. Python’s name-resolution rule: if any statement inside a function rebinds a name, that name is treated as local for the entire function body, regardless of textual order. So when `main()` reached `agent = O20Agent(custom_train_env, test_env, config, device)` on line ~ 270 in the `o2o` branch, the local-binding promise had already been made by the import inside the unreached `offline` branch — and the local had not yet been assigned. Result: `UnboundLocalError: cannot access local variable 'O20Agent' where it is not associated with a value` at the first instruction of the `o2o` phase.

Detected by. First O2O smoke test (`run_o2o.py --phase=o2o`).

Fix. Delete the redundant inline import (`scripts/run_o2o.py` line 309 at HEAD now contains a docstring comment instead). One line.

Integrity impact. Pre-fix, the `o2o` phase could not run at all. Any prior milestone result attributed to the O2O pipeline either came from the `phase=offline` branch (which doesn’t trigger the shadow) or was never produced. This bug is interlocked with Bug 3 below: even after fixing the import, the `o2o` branch crashed on its first online step. Together, Bugs 2 and 3 indicate the `o2o` phase had not been end-to-end exercised in CI before the Phase 2 prep work.

A.3 Bug 3: `transfer_to_online` left the SAC env in a terminal state

File. `hybrid_rl/agents/o2o_agent.py:142--164`.

Pre-fix behavior. `O20Agent.__init__` calls `self.sac_agent = SACDirichlet(train_env, config, device)` (line 84), and `SACDirichlet.__init__` immediately runs `self._obs, _ = env.reset()` (`online_rl/agents/sac` to cache the first observation. Subsequently, `agent.load_offline_data(...)` calls `self.offline.buffer.load_from_env(...)` — which shares the same `env` object — and rolls it forward to populate the offline buffer, leaving `train_env.t` at an exhausted/terminal index. The cached `sac_agent._obs` is now stale.

When `transfer_to_online` finishes copying parameters and the next `sac_agent.collect_step()` runs, line 147 of `sac_dirichlet.py` calls `self.env.step(action)` on the still-exhausted `env`, which raises `RuntimeError: Episode is done. Call reset() before stepping. (core/envs/portfolio_env.py:142)`.

Detected by. First successful run past Bug 2: the O2O smoke crashed immediately on entering Phase 2.

Fix. Append four lines to `transfer_to_online`:

```
self.sac_agent._obs, _ = self.sac_agent.env.reset()
self.sac_agent._episode_start = True
self.sac_agent._episode_return = 0.0
self.sac_agent._gru_hidden = self.sac_agent.regime_encoder.init_hidden(
    1, self.sac_agent.device
)
```

This re-anchors the SAC agent on a fresh post-reset `env` state, matching the contract the rest of `collect_step` expects.

Integrity impact. Same as Bug 2: pre-fix, the `o2o` phase could not make a single online gradient step. Any reported O2O result must have post-dated this fix.

A.4 Bug 4: run_o2o.py o2o-phase ignored cql_weight — the centerpiece bug

File (pre-fix). `scripts/run_o2o.py:307--326`, an inline online-phase loop that called

```
m.update(agent.sac_agent.update_critic(mixed))
```

without forwarding any `cql_weight` argument.

File (post-fix). `scripts/run_o2o.py:307--312`, replaced by a call to

```
o2o_history = agent.finetune_online(n_online)
```

Failure mode. `SACDirichlet.update_critic`’s signature is `def update_critic(self, batch, cql_weight: float = 0.0)` (`online_rl/agents/sac_dirichlet.py:173`). The pre-fix inline loop never passed the kwarg, so the critic was updated with `cql_weight=0.0` for the entire online phase — i.e., **no conservatism** in any iteration, regardless of the `--adaptive-conservatism` CLI flag that we added in this branch (and regardless of the original config’s `cql_alpha=5.0`).

The full `O2OAgent.finetune_online` method (`hybrid_rl/agents/o2o_agent.py:204--275`) implements the adaptive schedule:

- Tracks recent online regime samples (lines 225–241).
- Computes `cql_w = self._compute_adaptive_cql_weight(h_online, var_online)` via $\alpha_{\text{CQL}} \cdot \sigma(\eta \cdot \text{KL})$ (line 247, formula at lines 191–200).
- Calls `update_critic(mixed_batch, cql_weight=cql_w)` (line 258) — this time with the kwarg.

This entire pipeline existed in the codebase but was *never invoked* from the CLI: `run_o2o.py` rolled its own (broken) inline replacement.

Detected by. Reading both files side-by-side after Bug 3 was fixed, to figure out where the `--adaptive-conservatism` flag would actually take effect. It didn’t. The flag was a no-op until we swapped the inline loop for `agent.finetune_online`.

Fix. Replace the inline loop (and add metrics + cql-trajectory save): `scripts/run_o2o.py:307--356`. The new code calls `finetune_online`, which is the only path that respects the `adaptive-conservatism` config flag.

Integrity impact — read this carefully. Pre-fix, every o2o phase run, regardless of CLI flags, was a “fully-online fine-tune from offline init with zero conservatism” — i.e., it neither was the *adaptive O2O* condition described in the proposal nor the *naive fine-tune* condition in the four-way comparison nor anything in between. It was a third, different condition that the experimental design had no name for. Therefore: **the milestone’s planned adaptive-O2O vs naive-fine-tune comparison was silently a degenerate-case-vs-degenerate-case comparison until this fix.** All Phase 2C results post-date this fix; any prior “adaptive O2O” numbers from earlier branches must be relabeled or discarded. We surface this in Sec. 5 (Adaptive-O2O bug fix paragraph) rather than retroactively rewriting the milestone.

A.5 GRPO loss implementation scaffolding

Implementation details elided from the inline loss derivation (Sec. 3.2, Eq. 5); load-bearing for reproduction but not for the formulation contribution.

Importance-ratio hard-clamp. The log-ratio $\log \rho^{(i)} = \log \pi_\theta(a^{(i)} | s) - \log \pi_{\theta_{\text{old}}}(a^{(i)} | s)$ is clamped to $[\log 10^{-3}, \log 10^3]$ before exponentiation (`online_rl/agents/grpo.py:79--80, 145--148`). This is belt-and-suspenders against the first-epoch divergence that PPO clipping alone admits: when the policy moves far in a single epoch, ρ can blow up before the surrogate clip is consulted, and the exponentiation overflows; the log-domain clamp prevents NaN propagation through the gradient.

Closed-form Dirichlet→Dirichlet KL. The KL anchor term $D_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}})$ uses the closed-form Dirichlet→Dirichlet KL (`online_rl/agents/grpo.py:164--166`); no Monte-Carlo estimate. The reference actor’s parameters are frozen for the entire run and never re-snapshotted — a stronger anchor than RLHF-style periodic refreshes, chosen so that the policy cannot drift unboundedly under the small-batch financial signal-to-noise ratio.

Reference-snapshot mechanics. The reference policy π_{ref} is the actor weights at trainer construction time, i.e. *after* any optional offline warm-start load (`online_rl/agents/grpo.py:228`; ref is a frozen `deepcopy` of the actor immediately following the `actor.load_state_dict(...)` block in `scripts/train_grpo.py`). In the offline warm-start configuration, the KL term then reads as “do not drift far from what the offline pretraining converged to,” which is the standard O2O semantic.

Table 4: Phase 2A test metrics on the leaky pipeline. Behavior-anchored algorithms (BC, AWAC, CQL, IQL) collapse to the equal-weight basin (Sharpe ≈ 0.94 , turnover ≤ 0.005) under friction; pure Q -maximizing algorithms with weak anchoring (TD3+BC, BCQ) escape the basin and produce implausibly high Sharpe by exploiting the in-observation reward signal. Equal-weight reference: Sharpe 0.953. Mean $\pm$ std across 3 seeds.

algorithm	class	test Sharpe	test turnover
TD3+BC	Q -max + soft BC penalty	$+6.785 \pm 0.628$	0.644
BCQ	Q -max + VAE candidate filter	$+3.905 \pm 0.232$	0.263
BC	pure imitation	$+0.941 \pm 0.006$	0.003
AWAC	adv-weighted regression	$+0.942 \pm 0.011$	0.004
CQL (vanilla)	Q -max + conservative penalty	$+0.945 \pm 0.001$	0.0003
IQL ($\lambda=0.001$)	expectile + adv-weighted	$+0.943 \pm 0.002$	0.003

A.6 Cross-bug dependencies and CI implications

Bugs 2, 3, and 4 are stacked: fixing 2 is a prerequisite to even encountering 3, and fixing 3 is a prerequisite to encountering 4. The fact that all three lived in `run_o2o.py`'s `o2o` branch and were discovered in sequence by a single five-minute smoke test indicates the `o2o` phase had not been exercised end-to-end in the existing CI. We added no new CI step for this phase (Phase 2 budget did not allow it), but recommend a smoke-level integration test (`tests/test_o2o_pipeline_smoke.py`) that runs `--phase=o2o --n_offline_updates=10 --n_online_steps=20` as a follow-up before any extension of this work. The MomentumPolicy bug (1) is independent of the O2O cluster but has the same flavor: a unit test for behavior-policy diversity would have caught it in seconds.

B Leak detection and the limits of behavior anchoring

B.1 The leak

The offline data-utility `core/envs/data_utils.py:compute_features` computed today's per-asset log-return as feature index 0 (`features[t, asset, 0] = log_return[t]`, line 138 pre-fix) and its caller built the reward target as `forward_returns = np.expml(log_returns)` (line 268 in `make_train_test_envs` and line 372 in `make_train_val_test_envs`). After the caller's transform, `features[t, i, 0] = log(1 + forward_returns[t, i])` for every asset i — the same financial signal at the same index t . The agent's observation at decision time t therefore contained the return that its action would earn at step t , violating the documented PortfolioEnv contract that *forward_returns[t] is the return realized after acting on features[t]* (`core/envs/portfolio_env.py:30--31`).

The contemporaneous online-data utility `features/feature_engineering.py:build_features` is causal: `forward_returns[:-1] = prices_arr[1:] / prices_arr[:-1] - 1.0` (line 220) shifts forward returns by one step so that *forward_returns[t]* is the return between *prices[t]* and *prices[t + 1]*, with the final row trimmed. Phase 2B (online-only baselines) and Phase 2D (GRPO group-size ablation) load the causal datasets `datasets/real*.npz` produced by the online utility and were therefore never affected by the leak. Phase 2A (offline-only baselines) and the original Phase 2C (offline-to-online, killed mid-flight) consumed the leaky utility and were affected.

B.2 Differential exploitation on the leaky pipeline

Six offline-RL algorithms trained on the same leaky buffer and the same friction $\lambda = 0.001$ for 20,000 updates, evaluated on the same 1,063-day test window (Table 4).

A naïve read of Table 4 would conclude that TD3+BC and BCQ have found a strong real signal. Two observations falsify this read:

1. Sharpe > 6 on a 4-year backtest with no oracle, on an 8-asset universe at daily frequency, is essentially unattainable; a deterministic strategy that places 100% on the next-day winning asset earns roughly 1%/day at $\sim 1\%$ daily volatility, annualizing to Sharpe ≈ 7 . The TD3+BC number lands in exactly that regime.

Table 5: Phase 2A causal sanity check, $n=3$ seeds per row. The four behavior-anchored algorithms cluster tightly at Sharpe ≈ 0.94 on the causal pipeline, mirroring the leaky-pipeline behavior. Every cell lands inside the pre-registered $[0.85, 1.05]$ leak-invariance band. Equal-weight reference: Sharpe 0.953.

algorithm	test Sharpe (mean $\pm$ std)	range	turnover	cum. return
BC	$+0.935 \pm 0.002$	$[0.934, 0.937]$	0.0025	+0.321
AWAC	$+0.940 \pm 0.004$	$[0.935, 0.942]$	0.0022	+0.324
CQL_vanilla	$+0.952 \pm 0.001$	$[0.951, 0.953]$	0.00005	+0.331
IQL	$+0.935 \pm 0.009$	$[0.929, 0.945]$	0.0023	+0.317

Table 6: 56-d vs 216-d IQL on $\lambda=0.001$ test window. The 56-d configuration sits in the equal-weight basin; the 216-d configuration escapes the basin and engages in active trading at net Sharpe cost.

configuration	seeds	test Sharpe	turnover	cum. return	max DD
56-d IQL	3	$+0.935 \pm 0.009$	0.0023	+0.317	0.119
216-d IQL	1	+0.398	0.206	+0.120	0.167

2. CQL_vanilla shares TD3+BC’s Q -maximizing structure but adds an explicit conservative- Q penalty that suppresses out-of-distribution actions. Its Sharpe matches BC (0.945 vs 0.941) and its turnover is the lowest in the table (0.0003). The conservative penalty is the load-bearing differential: it is exactly the mechanism designed to block OOD- Q exploitation, and its presence pins CQL to the behavior-anchored cluster.

The pattern is the algorithm-class signature of label leakage: the algorithms that *can* leave the behavior-policy support (unconstrained Q -maximization with weak anchoring) read tomorrow’s reward off today’s observation and concentrate on it; the algorithms constrained to the behavior support cannot exploit a signal the behavior policy doesn’t generate.

B.3 Causal-pipeline confirmation

The fix (§B.5) shifts forward-returns by one step. We re-ran the four behavior-anchored offline algorithms on the causal pipeline at $\lambda = 0.001$, 3 seeds each, 20,000 updates, 1,061-day test window¹. Results in Table 5.

The Sharpe numbers tighten by an order of magnitude between Tables 4 and 5 for the four behavior-anchored algorithms, and every cell lands inside the pre-registered $[0.85, 1.05]$ band for the leak-invariance claim. We do not re-run TD3+BC or BCQ on the causal pipeline; their absence from the four-way comparison (Sec. 4.4) is intentional, and the leaky results in Table 4 are the contribution we want from those rows.

B.4 Scope condition: leak-invariance is feature-space-specific

The behavior-anchored leak-invariance argument (§B.2) reads naturally as a method-class property, but our experiments only directly support the claim *conditional on the 56-d feature space* used by the offline RL pipeline (`core/envs/data_utils.py:compute_features`: per-asset log-return + rolling vol + RSI + MACD + Bollinger %B + previous weights, $8 \times 6 + 8 = 56$ dims). To probe how far this generalizes, we trained a one-off IQL on the richer 216-d feature space used by `features/feature_engineering.py:build_features` (lagged log-returns + rolling volatility + multi-window momentum, no previous weights, 216 dims), originally to provide a feature-compatible warm-start source for GRPO. The result is informative on its own.

Two interpretations of Table 6 are empirically distinguished by the training-time dynamics, captured in `writeup/216d.iql_dynamics.md`. The 216-d IQL’s val Sharpe *transitions through the equal-weight basin*: at update step 1000, val Sharpe is +1.32 with turnover 0.004 (deep inside the basin); by step 5000, val Sharpe is +0.05 with turnover 0.18 (escaping); by step 19000, val Sharpe is -0.95 with turnover 0.33 (well outside, monotonically degrading). The IQL critic and value losses are essentially flat throughout (*advantage_mean* ≈ 0 , *weight_mean* ≈ 1.0): the drift is plain BC over-fitting to the 216-d feature space’s idiosyncratic structure, not a critic-driven escape toward high- Q regions.

¹The post-fix window is two days shorter than the pre-fix window: one row is trimmed from the train-test slice because `forward_returns[T - 1]` is undefined.

The cleaner statement is therefore: **leak-invariance is feature-space-specific in its dynamics**. On the 56-d feature space the equal-weight basin is a fixed point of the 20,000-update training trajectory; on the 216-d feature space it is a transient state at roughly step 1000, after which BC over-fitting drives the policy into a feature-driven active-trading regime. Both behaviors satisfy the *absence-of-implausible-Sharpe* signature that defines the leak-invariance claim — we never see the TD3+BC +6.8 on either feature space — but only the 56-d case lands at the basin under the default training budget.

This connects to the broader friction-collapse phenomenon documented in the milestone via the IQL λ -anchor: friction punishes active trading regardless of whether the policy’s degrees of freedom come from the conservatism schedule (milestone λ -sweep) or from the feature space (this section). Both are manifestations of the same mechanism — transaction costs cap the value of any non-equal-weight allocation drift on this 8-asset universe at $\lambda=0.001$ — and behavior-anchored offline RL surfaces it along whichever degree of freedom the configuration affords.

The Phase 2C “GRPO with offline warm-start” condition was originally planned around a 216-d IQL warm-start source; given the basin-transient character of 216-d IQL at the chosen training budget, we leave GRPO warm-start to future work and surface the 216-d/56-d differential as the substantive finding in its place (Sec. 4.4).

B.5 The one-line fix

Both `make_train_test_envs` and `make_train_val_test_envs` in `core/envs/data_utils.py` now build forward returns and features as:

```
flat_features_full = _build_flat_features(features, macro_arr, sentiment_arr)
fwd_full = np.expm1(log_returns).astype(np.float32)
# Causal shift: forward_returns[t] is the return realized AFTER the
# agent acts on features[t] (matches PortfolioEnv contract). Pre-fix,
# forward_returns[t] was today’s return AND features[t,:,0] was the
# same log-return -- leaking the reward into the observation.
flat_features = flat_features_full[:-1]
forward_returns = fwd_full[1:]
```

Diff: 20 lines added, 4 removed, applied to two functions (git commit 65d4040 on branch `phase2-leak-fix`). The online-data utility `features/feature_engineering.py:219--220` already implemented this contract; the patch brings the offline utility into agreement.

B.6 Methodological observation: differential as diagnostic

The clean diagnostic for label leakage in an offline-RL benchmark is the *spread between behavior-anchored and Q -maximizing algorithms on the same dataset*. With a well-constructed dataset:

- Imitation-anchored methods (BC, AWAC, IQL with high β -anchor, CQL with strong pessimism) approximate the behavior policy’s mean performance. They cannot dramatically exceed it, because their objective forbids drift from behavior support.
- Q -maximizing methods with weak anchoring (TD3+BC, BCQ) modestly exceed the behavior policy if the critic generalizes. They cannot *dramatically* exceed it on a clean dataset, because OOD- Q exploitation is exactly what conservative- Q families are designed to suppress.

A large gap between the two classes is the load-bearing observation for “the dataset has a label leak.” Specifically, on this 8-asset universe at $\lambda = 0.001$:

- TD3+BC $\approx$ BC $\approx$ EW: friction collapse / no exploitable signal. (This is what the causal pipeline produces.)
- TD3+BC $>$ BC by a factor > 2 with non-trivial turnover, AND CQL_vanilla $\approx$ BC: **label leak**. (This is what the leaky pipeline produced.)

The differential is robust to the absolute level of either method: neither “TD3+BC is high” nor “BC is low” alone would have flagged the bug. The combination, with CQL_vanilla providing the control row that distinguishes “no signal” from “leakage,” is what made the diagnosis falsifiable.

We recommend that any offline-RL benchmark builder run TD3+BC (or any pure Q -maximizing baseline with low BC weight) alongside a strong behavior-anchored baseline (BC or AWAC) and a

conservative- Q baseline (CQL-vanilla) on every dataset, and treat a $> 2\times$ Sharpe gap with the conservative baseline anchored to BC as a STOP condition for label-leak inspection. The pre-fit Phase 2A grid in this work was, in hindsight, an effective leak detector before it was a method comparison; we surface this experience in the hope that it generalizes to other offline-RL pipelines.

References

- Gaon An, Seungyong Moon, Jang-Hyun Kim, and Hyun Oh Song. Uncertainty-based offline reinforcement learning with diversified q-ensemble. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- Lili Chen, Kevin Lu, Aravind Rajeswaran, Kimin Lee, Aditya Grover, Misha Laskin, Pieter Abbeel, Aravind Srinivas, and Igor Mordatch. Decision transformer: Reinforcement learning via sequence modeling. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- DeepSeek-AI. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*, 2024.
- Yue Deng, Feng Bao, Youyong Kong, Zhiquan Ren, and Qionghai Dai. Deep direct reinforcement learning for financial signal representation and trading. *IEEE Transactions on Neural Networks and Learning Systems*, 28(3):653–664, 2017.
- Hanze Dong, Wei Xiong, Bo Pang, Haoxiang Wang, Han Zhao, Yingbo Zhou, Nan Jiang, Doyen Sahoo, Caiming Xiong, and Tong Zhang. RLHF workflow: From reward modeling to online RLHF. *arXiv preprint arXiv:2405.07863*, 2024.
- Miroslav Dudík, Dumitru Erhan, John Langford, and Lihong Li. Doubly robust policy evaluation and optimization. In *Statistical Science*, 2014.
- Jakob Foerster, Gregory Farquhar, Triantafyllos Afouras, Nantas Nardelli, and Shimon Whiteson. Counterfactual multi-agent policy gradients. In *AAAI Conference on Artificial Intelligence*, 2018.
- Scott Fujimoto and Shixiang Shane Gu. A minimalist approach to offline reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- Scott Fujimoto, David Meger, and Doina Precup. Off-policy deep reinforcement learning without exploration. In *International Conference on Machine Learning (ICML)*, 2019.
- Zhengyao Jiang, Dixing Xu, and Jinjun Liang. A deep reinforcement learning framework for the financial portfolio management problem. *arXiv preprint arXiv:1706.10059*, 2017.
- Thorsten Joachims, Adith Swaminathan, and Maarten de Rijke. Deep learning with logged bandit feedback. *International Conference on Learning Representations*, 2018.
- Ilya Kostrikov, Ashvin Nair, and Sergey Levine. Offline reinforcement learning with implicit q-learning. In *International Conference on Learning Representations (ICLR)*, 2022.
- Aviral Kumar, Aurick Zhou, George Tucker, and Sergey Levine. Conservative q-learning for offline reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- Xiao-Yang Liu, Hongyang Yang, Qian Chen, Runjia Zhang, Liuqing Yang, Bowen Xiao, and Christina Dan Wang. FinRL: A deep reinforcement learning library for automated stock trading in quantitative finance. *arXiv preprint arXiv:2011.09607*, 2020.
- Yecheng Jason Ma, Andrew Yan, Joey Hejna, Marco Pavone, Andrea Bajcsy, Chelsea Finn, Lerrel Pinto, and Osbert Bastani. Conservative and adaptive penalty for model-based safe reinforcement learning. *arXiv preprint arXiv:2207.07276*, 2022.
- John Moody and Matthew Saffell. Learning to trade via direct reinforcement. *IEEE Transactions on Neural Networks*, 12(4):875–889, 2001.
- Ashvin Nair, Abhishek Gupta, Murtaza Dalal, and Sergey Levine. AWAC: Accelerating online reinforcement learning with offline datasets. *arXiv preprint arXiv:2006.09359*, 2020.

- Dean A. Pomerleau. Efficient training of artificial neural networks for autonomous navigation. *Neural Computation*, 3(1):88–97, 1991.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Sirui Zhao and Tim Pearce. Adaptive online replanning with diffusion models. *arXiv preprint arXiv:2210.06463*, 2022.
- Yi Zheng, Jianxiong Li, Dongjie Yu, Yinan Yang, Sergey E. Lin, Hang Zhao, Boyi Liu, and Xianyuan Zhan. Adaptive behavior cloning regularization for stable offline-to-online reinforcement learning. *arXiv preprint arXiv:2210.13846*, 2023.